

Purity and allocation in the BoCa mechanisation

A short note on the `Extensions/Purity` branch

This note summarises an extension of the Lean mechanisation of *From Linearity to Borrowing* (Wagner et al., OOPSLA 2025), prompted by Matsushita and Ishii’s *Pure Borrow* (PLDI 2026), whose §7 observes that the BoLo metatheory does not address the functional behaviour of borrowing programs (issue #1). All results below are machine-checked, use only `propext`, `Classical.choice` and `Quot.sound`, and leave every printed statement unchanged. Lean names refer to `Extensions/Purity/` and `Support/` on branch `purity`; `Extensions/Purity/README.md` has the full account.

1 Runs as a parameter of wp

The printed `wp` (row 5.33) asks for *some* run to a value, so adequacy says that some run of a well-typed program is safe and frees what it allocates. The proof of [TR] 6.141 uses only that the chosen location is fresh: `wpTS_alloc_any` proves the allocation rule at every location missing from the heap.

We state `wp` at a *class of runs* `RunRel`, closed under the operations its rules build runs with, and indexed so that a run plugged into a context K may be required to do more (`shift`). Only eight constructions build runs (value, head step, allocation, free, the two loads, store, bind); every other rule of the typed world passes the run through unchanged. The Fundamental Property holds at every class (`fundamentalR`). The printed `wp`, `fundamental` and `adequacy` are the instance at every run, and are now corollaries (§12.74 of `docs/adjudications.md`). Two further instances:

- **Allocation policies.** A policy picks a heap-fresh location as a function of the heap. `adequacyPol`: for *every* policy, the unique run of a closed well-typed program is safe, terminates and frees everything. Least-free allocation, which reuses locations, is an instance; the printed `adequacy` is derived from it.
- **Fresh runs.** Each allocation avoids every location the configuration names (heap, term and stored values) and a finite set N ; plugging into K adds the locations of K to N . `adequacyFresh` is adequacy with fresh allocation.

2 Purity

Allocation is the machine’s only nondeterminism, and on fresh runs it is invisible up to a renaming of locations (`freshRun_det`, `fresh_run_local`).

Closed programs. `freshRunsExistClosed`: a closed well-typed program has a fresh run from the heap of every tagged typed world. Hence `closed_pure_every_run`: a closed program of plain-data type $(1, \oplus, \otimes)$ returns one location-free value on *every* run from *every* heap.

The pure fragment. A program whose arguments are plain data and `lmm` borrows and whose result is plain data (`PureTyping`) may allocate, mutate and free internally. From the heap μ of a tagged typed world, a fresh run gives μ back and returns a location-free v ; every run from μ , and every run from any heap agreeing with μ on what the arguments reach, returns v . This holds

- with the arguments in the printed relation, at argument types where no function type is read at the program’s view (`pure_result_every_run_ind`); there the relation does not depend on the run class at all (`vX_indep`);

- at *every* argument type, for arguments produced by a well-typed program (`arising_producer_pure`), following the reading that §6 quantifies over resources that arise from the calculus.

Within a borrow’s lifetime, nothing beneath an `imm` cell changes on the exhibited run (`wpTS_frozen`): the model’s deep immutability ([CONF] §5), at the level of the machine.

3 Where the boundary lies

Four small terms, each built in Lean, mark the edge of these statements. None is well typed; the first three are admitted by the semantic relation, and the typing is what excludes them.

- `peek`: in the relation at 1, but its run reads a location of the frame, so its behaviour depends on the allocator (`not_readFootprintSem`, `not_semPolicyRuns`). Well-typed programs read only what their arguments reach or they allocate (`readFootprintSyn`).
- `staleL`: location-free and in the relation, but it reads through a freed and reallocated name, so it has no fresh run (`staleL_no_fresh`).
- `staleClosure` ($\lambda_. \text{staleL}$): admitted at $1 \multimap 1$ by the relation at every run but not at the fresh runs (`vX_lolli_depends`).
- `staleH`: a stale name kept in a cell rather than the term (`not_termLiveSuffices`).

4 Open

Each item is not derived; no obstruction to it is verified.

- The pure fragment with arguments in the printed relation at types such as `Imm @a (Mut @b (1  $\multimap$  1))`, where a `Mut` cell’s stored predicate mentions `wp`. Such arguments are covered when they arise from well-typed code (`Arises.mut_pred`).
- Arguments produced under an arbitrary (non-fresh) allocator. This needs an invariant along runs, a run-time store typing with owned, lent and moved-out cells; the README sketches it. The `withswap` window, where a lent cell names a payload the callback may free, is recorded but not machine-checked.
- A state-monad denotation for programs with `Mut` borrows or owned references in their interface, of which the pure fragment would be the empty-state case.